# JustCmul8 Viva Questions

*Short questions and simple answers, based on the project report*

## 1. About the Project

**Q1. What is your project?**

JustCmul8 is a website where you can build and run a simulation without writing code. You drag boxes on a screen, join them, fill a form and press Run.

**Q2. What is Discrete Event Simulation (DES)?**

It is a way to copy a real system on a computer. The clock jumps from one event to the next event, like a customer arriving or a job finishing.

**Q3. Why the name JustCmul8?**

It sounds like “Just Simulate”. The idea is that simulation should be that easy.

**Q4. Who will use it?**

Students, teachers, researchers and small business owners.

**Q5. Give one real example.**

A bank wants to know if one more counter will cut the waiting line. We can test it on the computer first, without spending money.

**Q6. What problem does it solve?**

Three problems. Paid tools are very costly, free tools need coding, and most tools must be installed. Our tool is free, needs no code and runs in the browser.

**Q7. Name some existing tools.**

AnyLogic, Arena and Simul8 are paid. SimPy is free but needs Python coding. Visio and Lucidchart only draw diagrams, they do not run them.

**Q8. What is special in your project?**

It runs a real Python engine (SimPy) inside the browser. So the answers are real, not a fake animation.

**Q9. What are the limits of your project?**

Model size depends on the browser memory. Monte Carlo runs one after another. The first run needs a small download. Saving, AI and collaboration need internet. There is no mobile app.

## 2. Requirements and Feasibility

**Q10. What are functional requirements?**

What the system must do. For example, login, build a graph, run a simulation, show results. We have 27 of them (FR 01 to FR 27).

**Q11. What are non functional requirements?**

How well the system must work. For example, speed, security, correctness and ease of use. We have 11 of them.

**Q12. Who are your user types?**

Registered user (owner), collaborator and guest viewer. The guest can only view through a share link.

**Q13. Which feasibility studies did you do?**

Technical, economic, operational, schedule, and legal and ethical.

**Q14. What was the biggest technical risk?**

Running Python SimPy inside the browser. We solved it with Pyodide, which is Python made to run in WebAssembly.

**Q15. Why is it economically feasible?**

The tools are open source and the cloud services have free plans. The simulation runs on the user's computer, so server cost does not grow with use.

**Q16. What does the user need to run it?**

Only a modern browser like Chrome, Edge, Firefox or Safari. Minimum 4 GB RAM. Nothing to install.

**Q17. What are user personas?**

Sample users we designed for: a student, a lecturer and a small business owner.

## 3. Planning and Gantt Chart

**Q18. Which development model did you use?**

Iterative and incremental model. We built and tested one module at a time.

**Q19. Why not waterfall?**

In waterfall we would find the engine risk very late. We wanted to test the risky part first.

**Q20. Why not pure Agile?**

Our requirements were stable and we needed proper documents for college.

**Q21. Why not Spiral?**

It is too heavy for a team of two people.

**Q22. How many activities are in your plan?**

27 activities, from project start to deployment.

**Q23. What is a Gantt chart?**

A chart that shows each task as a bar on a calendar. It shows when a task starts, how long it takes and which tasks overlap.

**Q24. What time does your Gantt chart cover?**

July 2026 to March 2027. The main plan was 24 weeks, July to December 2026. Then we added January to March 2027 for extra testing, bug fixing and two future scope items.

**Q25. What did you add in January to March?**

Regression testing, production bug fixing, Parallel Monte Carlo, reproducible seeded runs, testing of these, and a final release v1.1.

**Q26. What do the colours in the Gantt chart mean?**

Blue is analysis, purple is design, red is development, orange is testing, grey is documentation and green is review and deployment.

**Q27. What are the red diamonds?**

They are milestones, an important point where a task is finished.

**Q28. Why did the Simulation Engine start so early?**

It was the riskiest part and many modules depend on it. Starting early gave time to fix problems.

**Q29. Which milestone is most important?**

M4, the engine proof of concept. It proved Python can run in the browser.

**Q30. What is an Activity Network Diagram?**

It shows the order of the work, which tasks run together and where decisions are made. The Gantt chart shows when, this shows in what order.

**Q31. What is the critical path?**

The longest chain of tasks. If any task on it is late, the whole project is late. Ours goes through requirements, analysis, database design, backend, integration, testing, UAT and deployment.

**Q32. What is float or slack?**

Extra time a task has. Tasks like Template Gallery can be a little late without delaying the project.

**Q33. Which project tool did you use?**

Jira. We used a Scrum board, two week sprints, epics for modules and bug tickets.

## 4. Modules

**Q34. How many modules are there?**

12 modules. Each team member made 6.

**Q35. Name the modules.**

Platform and Data Layer, Visual Graph Editor, Node Configuration Panel, Simulation Engine, AI Assistant, AI System Optimiser, KPI Dashboard, Monte Carlo, Digital Twin, Template Gallery, Report and Share, Real Time Collaboration.

**Q36. The synopsis had 11 modules. Why 12 now?**

We joined login, project and profile into one module. Then the AI Optimiser, Monte Carlo and Digital Twin became separate modules because they grew big.

**Q37. How many node types are there?**

15 node types. The main ones are source, queue, resource, decision and sink.

**Q38. What does a source node do?**

It creates new entities, like customers arriving.

**Q39. What does a sink node do?**

It is the exit. The entity finishes its journey there.

**Q40. What does a decision node do?**

It sends an entity to one path or another based on a probability, like 70 percent to one side and 30 percent to the other.

**Q41. How many domains do you support?**

Six: human queues, vehicle traffic, liquid and material flow, manufacturing, logistics, and network signals.

**Q42. What does the Node Configuration Panel do?**

It is a side panel to set values like arrival rate, service time, capacity and cost for the selected node.

**Q43. What does the AI Assistant do?**

You type in plain English and it makes a model for you. You see a preview first and the model changes only when you accept.

**Q44. What does the AI Optimiser do?**

After a run it finds the slowest node and suggests one fix. You can apply it with one click.

**Q45. What is the KPI Dashboard?**

It shows results like utilisation, waiting time, throughput, queue length, bottleneck and a health score from 0 to 100.

**Q46. What is Monte Carlo in your project?**

We run the same model 10, 20, 50 or 100 times and show the average with a 95 percent confidence range.

**Q47. What is the Digital Twin?**

An animation that replays the run. You can see entities moving and lines building up.

**Q48. What is the Template Gallery?**

Ready made starter models for all six domains, so a new user does not start from a blank screen.

**Q49. What can be exported?**

Graph as JSON, results as CSV, Python code as a Jupyter notebook, and a PDF report.

**Q50. What is real time collaboration?**

Many users can work on the same project together. They see each other's cursors and changes, like in Figma or Canva.

## 5. Diagrams

**Q51. What is a Context Diagram?**

It is DFD level 0. It shows the whole system as one circle and the outside things that send or get data from it.

**Q52. Who are the external entities in your context diagram?**

Simulation Builder, Shared Link Viewer, Supabase Auth, and AI and Simulation Engine Services.

**Q53. Why is the database inside the boundary?**

Because our application sets it up and controls it.

**Q54. What is DFD level 1?**

It breaks the one process into smaller processes and shows the data stores. Ours has 7 processes, P1 to P7.

**Q55. Name the 7 processes.**

Account and Access, Project Management, Graph Design, Simulation Execution, AI Assistance, Sharing and Export, and View and Results.

**Q56. What is an ER Diagram?**

It shows the data tables and how they are related. We used Chen notation: rectangle for entity, ellipse for attribute, diamond for relationship.

**Q57. Give one relationship from your ERD.**

One User has one Profile. One User owns many Projects. One Project has many simulation runs.

**Q58. Why is the bottleneck saved in the run table?**

So an old run keeps its result. We do not need to calculate it again.

**Q59. What is a Class Diagram?**

It shows the classes, their attributes, methods and links. Example classes are User, Project, SimNode, SimEdge and SimulationRun.

**Q60. Why are node settings stored as JSON?**

15 node types have different settings. JSON lets us store all of them in one class without many empty columns.

**Q61. What is a State Transition Diagram?**

It shows the states of a node while the simulation runs: IDLE, QUEUED, PROCESSING, COMPLETED, BLOCKED and FAILED.

**Q62. When is a node BLOCKED?**

When it has finished work but the next node is full. It waits until there is space.

**Q63. When is a node FAILED?**

When it is interrupted, pushed out by a higher priority job, or there is an error.

**Q64. What is a Use Case Diagram?**

It shows the users (actors) and what they can do with the system.

**Q65. Who are the actors?**

Simulation Builder and Shared Link Viewer. Supabase Auth, the SimPy engine and Gemini are external systems.

**Q66. Why did you use include and not extend?**

Include means the step always happens. For example, login always needs Supabase Auth.

**Q67. What is an Activity Diagram?**

It shows the flow of steps from start to end. Ours shows login, build graph, run, see results, AI help and export, using swimlanes.

**Q68. What are swimlanes?**

Columns that show who does each step, like the user, the system or the AI.

**Q69. What is a Component Diagram?**

It shows the software parts and how they depend on each other.

**Q70. What is a Collaboration Diagram?**

It shows objects and the numbered messages they send during one simulation run.

**Q71. What is a Deployment Diagram?**

It shows where the software runs: the user's browser, Vercel server and Supabase cloud.

**Q72. Explain your architecture.**

It has layers: presentation (screens), state (Zustand), domain services (code generator, analytics), execution (Web Worker with Pyodide) and database (Supabase).

**Q73. What is unusual in your architecture?**

The simulation runs on the user's computer, not on the server.

## 6. Database and Tables

**Q74. Which database do you use?**

PostgreSQL, through Supabase.

**Q75. How many tables are there?**

Five app tables: profiles, projects, simulation_runs, chat_history and project_shares. Users are kept in auth.users by Supabase.

**Q76. What is the primary key type?**

UUID, created by the database.

**Q77. What is Row Level Security (RLS)?**

A database rule that lets a user see only their own rows. Even if someone sends a request by hand, they get nothing.

**Q78. What is ON DELETE CASCADE?**

When a parent row is deleted, its child rows are deleted too. So deleting an account removes all its data.

**Q79. How does a share link stay safe?**

The link has a random token. The viewer can only call two small database functions. They never touch a table directly and can only read.

**Q80. What happens when a share link is revoked?**

We do not delete the row, we add a revoked time. Then the viewer sees a clear notice instead of an error.

**Q81. Where is the graph stored?**

In the projects table as one JSONB column called graph_json.

**Q82. How are passwords stored?**

Supabase stores only a salted hash. We never store plain passwords.

## 7. Technology Stack

**Q83. What is your frontend?**

Next.js 16, React 19 and TypeScript, with Tailwind CSS for styling.

**Q84. Why TypeScript?**

It catches type mistakes before the code runs.

**Q85. What is React Flow?**

A library for the drag and drop canvas with nodes and edges.

**Q86. What is Pyodide?**

Python made to run inside the browser using WebAssembly.

**Q87. What is WebAssembly?**

A fast code format that browsers can run at near native speed.

**Q88. What is SimPy?**

A free Python library for discrete event simulation.

**Q89. What is a Web Worker?**

A background thread in the browser. The simulation runs there, so the screen never freezes.

**Q90. What is Zustand?**

A small library to store the app state.

**Q91. What is Supabase?**

A cloud backend that gives us database, login and real time channels.

**Q92. Which AI do you use?**

Google Gemini, through a model router.

**Q93. What is the model router?**

It keeps a list of AI models. If one fails or is busy, it moves to the next one on its own.

**Q94. Where is the AI key kept?**

Only on the server. It is never sent to the browser.

**Q95. What is Pixi.js used for?**

To draw the Digital Twin animation fast using WebGL.

**Q96. What is Recharts used for?**

To draw the charts on the dashboard.

**Q97. How is the PDF report made?**

With jsPDF and html to image, inside the browser.

**Q98. Where is the app hosted?**

Frontend on Vercel. Database and login on Supabase.

## 8. Algorithms and Formulas

**Q99. How is the graph stored in memory?**

As an adjacency list. Each node keeps a list of its next nodes. It saves space because each node has only one or two links.

**Q100. How do you check if a graph is valid?**

We walk through the graph and mark visited nodes. Any node we cannot reach is reported before the run.

**Q101. What data structure holds the events?**

A priority queue made as a binary min heap. The earliest event comes out first. Insert and remove take O(log n) time.

**Q102. Which probability distributions are there?**

Five: exponential, uniform, normal, Poisson and deterministic.

**Q103. Why is the normal distribution cut at zero?**

Because a service time cannot be negative.

**Q104. What is utilisation?**

How much of the time a server is busy. For M/M/1 it is $\rho = \lambda / \mu$, arrival rate divided by service rate.

**Q105. What is throughput?**

How many entities finish per unit of time.

**Q106. What is a bottleneck?**

The node with the highest average waiting time. It slows down the whole system.

**Q107. What is Little's Law?**

L = λW. Average items in the system equals arrival rate times average time spent. We use it to check that a run is correct.

**Q108. How is the health score made?**

It is out of 100: completion 35, utilisation balance 30, stability 20 and tail waiting time 15.

**Q109. What is a 95 percent confidence interval?**

A range where the true value most likely lies. Formula: mean ± 1.96 × s / √n.

**Q110. Why only 10, 20, 50 or 100 runs?**

The range gets smaller with the square root of n. Four times more runs make the range half as wide.

**Q111. What is M/M/1?**

A queue with random arrivals, random service times and one server. M/M/c has c servers.

**Q112. What is Erlang C?**

A formula to find the waiting time in a queue with many servers.

## 9. Testing

**Q113. How did you test the project?**

In two parts. First we checked the engine against queueing theory. Then we tested the app features.

**Q114. Which types of testing did you do?**

Unit, structural, analytical verification, invariant, integration, security, cross browser and user acceptance testing.

**Q115. What is analytical verification?**

We compare the engine's answers with known maths formulas like M/M/1, M/M/3 and Little's Law.

**Q116. How many engine checks passed?**

All 28 out of 28. No result was more than 2.1 percent away from theory.

**Q117. Give one example.**

M/M/1 at ρ = 0.8: theory utilisation 0.8000, our result 0.7979. Theory wait 4.000 s, our result 4.005 s.

**Q118. What is invariant testing?**

Checking laws that must always be true, like nothing is lost or created. We used a hospital model and all 22 checks passed.

**Q119. Why did you run 5 replications?**

So a pass is not just luck from one run.

**Q120. What is a tolerance?**

How far the result can be from theory and still pass. We fixed it before testing.

**Q121. How many functional test cases are there?**

50 test cases, TC 01 to TC 50. Each one is linked to a requirement.

**Q122. What is negative testing?**

Giving wrong inputs on purpose, like capacity 0, empty graph or bad AI reply, to check the app handles them.

**Q123. How did you do security testing?**

We sent requests straight to the database. We tried to read another user's project and write with a share token. All were blocked.

**Q124. What is UAT?**

User Acceptance Testing. Real users try the app without help to see if they can get a result.

**Q125. What happened in the saturated test?**

Arrivals were double the service rate. The health score fell to 45.8 and Little's Law showed a backlog. So the app correctly shows a failing system.

**Q126. How were bugs tracked?**

As Bug tickets in Jira. Each was fixed, tested again and linked to its fix.

**Q127. What is traceability?**

Linking each requirement to the test that proves it.

## 10. Results and Performance

**Q128. What are the main results tabs?**

Executive Summary, Flow and Balance Check, Traffic Heatmap, Line Buildup Timeline, Station Counters, Person or Item Journey, Live Activity Log, and Cost and ROI.

**Q129. How fast is the engine?**

A run with about 28,700 entities finished in 3.65 seconds.

**Q130. Why is DES fast?**

It jumps from event to event and skips the empty time.

**Q131. What limits model size?**

Browser memory, not CPU speed.

**Q132. What did the hospital model show?**

With patients leaving, 4.3 percent left unserved but the system stayed stable with a health score of 91.

## 11. Cost Estimation

**Q133. What is COCOMO?**

Constructive Cost Model. It estimates effort and time from lines of code.

**Q134. Which COCOMO mode did you use and why?**

Organic mode, because we are a small team with stable requirements in a known area.

**Q135. How big is your code?**

22,192 lines, which is 22.19 KSLOC, in 91 files.

**Q136. What did basic COCOMO give?**

62.19 person months, 12 months and about 5 people. Cost about 24.88 lakh rupees.

**Q137. Why is that different from your real effort?**

Basic COCOMO assumes every line is typed by hand. We used AI coding agents and ready libraries like SimPy, React Flow and Supabase.

**Q138. What is the adjusted COCOMO?**

We took 25 percent of the effort: 15.55 person months, about 7 months and 2 to 3 people. This is close to our real work.

**Q139. What is Function Point Analysis?**

It measures size by features delivered, not by lines of code.

**Q140. What are the 5 FPA components?**

External Input, External Output, External Inquiry, Internal Logical File and External Interface File.

**Q141. What were your function points?**

UFP is 230, VAF is 1.22, so FP is 281. Effort about 21.6 person months.

**Q142. What is VAF?**

Value Adjustment Factor. VAF = 0.65 + 0.01 × TDI. Our TDI is 57.

**Q143. What would a company spend?**

About 6.7 lakh to 11.7 lakh rupees.

**Q144. What did you spend?**

About 1,000 to 19,500 rupees, because we used free tools and free plans.

## 12. Future Scope

**Q145. What will you add next?**

Parallel Monte Carlo, seeded runs you can repeat, pause and resume, roles like editor and viewer, and faster first loading.

**Q146. Any bigger future plans?**

Custom node plugins, agent based simulation, offline mode, a mobile app, cloud runs for very large models and linking to real IoT data.

## 13. Tricky Questions

**Q147. Why not write the simulation in JavaScript?**

It would be faster to build but it would be a copy, not a tested engine. SimPy is trusted by researchers and we proved it with theory checks.

**Q148. If it runs in the browser, why do you need a server?**

Only for login, saving, collaboration and to keep the AI key secret. The simulation itself does not use the server.

**Q149. Can the AI change my model by itself?**

No. It always shows a preview and waits for your approval.

**Q150. What if the AI gives a wrong graph?**

We check every graph first. Unknown nodes or loose edges are rejected.

**Q151. What if a user changes the frontend code to see other data?**

It will not work. Row Level Security in the database blocks it.

**Q152. Is your simulation result always the same?**

No, because it uses random numbers. That is why we use Monte Carlo and confidence intervals.

**Q153. What did you learn from this project?**

Planning the risky part first, testing with real maths, and keeping security in the database.

**Q154. What was your role? (answer for yourself)**

Khushi: platform and login, node panel, AI assistant, KPI dashboard, digital twin, report and share. Mohit: graph editor, simulation engine, AI optimiser, Monte Carlo, templates, collaboration.